پروژه درس مهندسی نرم‌افزار — گزارش جامع فاز اول (اسپرینت اول)

توسعه سامانه مدیریت وظایف (Todo App) با چارچوب اسکرام، اصول شی‌گرایی و SOLID

شناسنامه پروژه و اعضای تیم

- **نام درس:** مهندسی نرم‌افزار
- **عنوان پروژه:** سامانه مدیریت وظایف تحت وب (Todo App)
- **دوره:** فاز اول — اسپرینت اول (توسعه قابلیت‌های پایه)
- **طول دوره اسپرینت:** ۲ هفته
- **اعضای تیم توسعه:**
  - **رضا صفری:** مدیر محصول، توسعه‌دهنده فول‌استک و هسته اصلی (Product Owner & Full-Stack / Core Developer)
  - **امیرحسین میرزایی:** معمار سیستم و مهندس پایگاه‌داده (System Architect & Database Engineer)
- **استک فنی و استانداردهای مهندسی (Tech Stack):**
  - **Backend:** پایتون ۳.۱۰+ و فریم‌ورک FastAPI
  - **Database:** پایگاه داده SQLite به همراه SQLAlchemy ORM (پایداری کامل داده‌ها در دیسک محلی)
  - **Data Validation:** کتابخانه Pydantic V2 (اعتبارسنجی دوطرفه کلاینت/سرور و حذف فضاهای خالی با متد strip)
  - **Frontend & UI:** استاندارد HTML5، معنایی CSS3، مدرن با پالت رنگی سرمه‌ای کهکشانی، فیروزه‌ای و کهربایی، سایه‌زنی و لبه‌های سه‌بعدی پشتیبانی از Specular، و `user-invalid:`

با فونت فارسی (RTL) جاوااسکریپت ماژولار، تقویم خورشیدی و چیدمان راست‌چین ،`has:` Vazirmatn

- **Code Documentation:** مستندسازی و کامنت‌گذاری ساختاریافته تمامی فایل‌ها بر اساس استاندارد Color Block Commenting
- **Cross-Platform Execution:** راه‌انداز یک‌کلیکی ویندوز (`start.bat`) و اسکریپت هوشمند (`run.py`) انتخاب پورت آزاد و اجرای خودکار مرورگر
- **Automated Testing:** پوشش ۸ تست واحد و یکپارچگی خودکار) TestClient و Pytest فریم‌ورک (با موفقیت ۱۰۰٪)
- **Version Control:** با استراتژی شاخه‌بندی مهندسی و پیام‌های GitHub و میزبانی در (Git) گیت کامیت استاندارد

۱. فهرست محصول (Product Backlog) و تفکیک کار

فهرست محصول (Product Backlog) مخزنی پویا و اولویت‌بندی‌شده از تمام ویژگی‌ها و نیازمندی‌های شناخته‌شده سامانه مدیریت وظایف است که در قالب «داستان کاربر» (User Story) و با متد MoSCoW اولویت‌بندی شده و مسئولیت هر بخش به تفکیک مشخص گردیده است.

اولویت‌بندی نیازمندی‌ها (تکنیک MoSCoW)

- **Must Have (باید باشد):** قابلیت‌های حیاتی اسپرینت اول (ثبت وظیفه، مشاهده فهرست وظایف، اعتبارسنجی ورودی، ماندگاری در دیتابیس).
- **Should Have (بهتر است باشد):** قابلیت‌های مهم برای اسپرینت دوم (تغییر وضعیت انجام به اتمام‌رسیده، حذف و ویرایش وظایف).
- **Could Have (می‌تواند باشد):** قابلیت‌های اختیاری (دسته‌بندی، فیلتر کردن و جستجو).
- **Won't Have (در این فاز نخواهد بود):** احراز هویت پیچیده چندکاربره و اعلانات پیامکی/ایمیلی.

جدول تفصیلی آیتم‌های فهرست محصول (PBI)

شناسه	عنوان آیتم (داستان کاربر)	اولویت	اسپرینت هدف	مسئول اصلی (Assignee)	معیارهای پذیرش (Acceptance Criteria)
PB-01	**ثبت وظیفه جدید (Add Task):** به عنوان کاربر، می‌خواهم بتوانم یک وظیفه جدید با عنوان و توضیحات اختیاری ثبت کنم تا کارهای روزمره‌ام را فراموش نکنم.	Must	اسپرینت ۱	رضا صفری	۱. کاربر فرم ثبت وظیفه را ببیند. ۲. پس از ثبت، شناسه یکتا، وضعیت پیش‌فرض «انجام‌نشده» و زمان ایجاد به آن اختصاص یابد. ۳. فرم پس از ثبت با موفقیت خالی شود و پیام تایید نمایش یابد.
PB-02	**اعتبارسنجی ورودی (Input Validation):** به عنوان سامانه، می‌خواهم مانع ثبت وظایف با عنوان خالی یا نامعتبر شوم تا اطلاعات ذخیره‌شده معتبر و باکیفیت باشند.	Must	اسپرینت ۱	رضا صفری	۱. اگر عنوان خالی یا صرفاً فاصله (Whitespace) باشد، فرم ارسال نشود. ۲. پیام خطای واضح با دسترس‌پذیری بالا (ARIA) در کنار فیلد ظاهر شود. ۳. بک‌اند نیز با کد وضعیت ۴۲۲ یا ۴۰۰ خطای اعتبارسنجی را بازگرداند.
PB-03	**مشاهده فهرست وظایف (View Tasks):** به عنوان کاربر، می‌خواهم بتوانم لیست همه وظایف ثبت‌شده را مشاهده کنم تا از روند کارها مطلع شوم.	Must	اسپرینت ۱	رضا صفری	۱. وظایف در قالب کارت‌های تفکیک‌شده با عنوان، توضیحات، برچسب وضعیت و تاریخ خورشیدی نمایش یابند. ۲. ترتیب نمایش بر اساس جدیدترین زمان ثبت (نزولی) باشد. ۳. در صورت خالی بودن دیتابیس، پیام وضعیت خالی (Empty State) مناسب نمایش یابد.

شناسه	عنوان آیتم (داستان کاربر)	اولویت	اسپرینت هدف	مسئول اصلی (Assignee)	معیارهای پذیرش (Acceptance Criteria)
PB-04	**ماندگاری داده‌ها (Data Persistence):** به عنوان مدیر سیستم، می‌خواهم اطلاعات در دیتابیس پایدار ذخیره شود تا با بستن برنامه داده‌ها از دست نروند.	Must	اسپرینت ۱	امیرحسین میرزایی	۱. داده‌ها در یک فایل SQLite پایدار ذخیره شوند. ۲. با خروج از برنامه و اجرای مجدد سرور، تمام وظایف قبلی عیناً بازیابی شوند. ۳. ساختار جدول شامل `id`، `title`، `description`، `status` و `created_at` باشد.
PB-05	**تغییر وضعیت انجام وظیفه (Toggle Status):** به عنوان کاربر، می‌خواهم بتوانم وضعیت وظیفه را از «انجام‌نشده» به «انجام‌شده» تغییر دهم.	Should	اسپرینت ۲	رضا صفری	۱. اضافه شدن چک‌باکس یا دکمه تغییر وضعیت کنار هر وظیفه. ۲. به‌روزرسانی آنی وضعیت در پایگاه داده با متد PATCH. ۳. تغییر استایل وظیفه انجام‌شده (افکت خط‌خوردگی متن).
PB-06	**حذف یا ویرایش وظیفه (Delete/Edit):** به عنوان کاربر، می‌خواهم وظایف اشتباه را حذف یا عنوان آن‌ها را اصلاح کنم.	Should	اسپرینت ۲	امیرحسین میرزایی	۱. وجود دکمه حذف با پیام تایید (Confirm). ۲. امکان ویرایش عنوان و توضیحات وظیفه در سمت سرور و پایگاه داده.
PB-07	**فیلتر و جستجوی وظایف (Filter/Search):** به عنوان کاربر، می‌خواهم وظایف را بر اساس وضعیت یا کلمات کلیدی جستجو کنم.	Could	اسپرینت ۲	رضا صفری	۱. فیلتر نمایش بر اساس «همه»، «انجام‌شده» و «انجام‌نشده». ۲. کادر جستجوی زنده بر روی عنوان و توضیحات.

تفکیک کار و سهم مشارکت اعضا

- **رضا صفری (توسعه‌دهنده اصلی و فرانت‌اند / لاجیک):** مسئول آیتم‌های PB-01، PB-02، PB-03، PB-05 و PB-07؛ طراحی رابط کاربری مدرن با پالت سرمه‌ای کهکشانی، فیروزه‌ای و کهربایی و لبه‌های سه‌بعدی Specular، اعتبارسنجی دوطرفه ورودی‌ها، کنترلرها و اندپوینت‌های FastAPI، مدیریت دسترس‌پذیری ARIA، تبدیل تقویم میلادی به خورشیدی، مستندسازی کدها با الگوی Color Block و ساخت اسکریپت‌های اجرای خودکار (`start.bat` و `run.py`).
- **امیرحسین میرزایی (معمار سیستم و مسئول پایگاه داده):** مسئول آیتم‌های PB-04 و PB-06؛ طراحی ساختار و مدل‌های داده‌ای ORM در SQLAlchemy، پیاده‌سازی کلاس مخزن داده `SQLiteTaskRepository`، تضمین پایداری داده‌ها بر روی دیسک، مدیریت نشست‌های دیتابیس و بررسی معماری SOLID.

۲. فهرست اسپرینت اول (Sprint Backlog) و Definition of Done

هدف اسپرینت (Sprint Goal)

توسعه نسخه اولیه، پایدار و چندسکویی از سامانه مدیریت وظایف تحت وب با رعایت اصول شی‌گرایی و SOLID، به‌گونه‌ای که کاربر بتواند با یک کلیک برنامه را اجرا کرده، وظایف جدید را با اعتبارسنجی معتبر ثبت کند، کلیه وظایف ذخیره‌شده در پایگاه داده SQLite را در رابط کاربری فارسی و مدرن مشاهده نماید و اطلاعات پس از راه‌اندازی مجدد سیستم کاملاً حفظ شود.

جدول شکست ساختار وظایف فنی (Task Breakdown)

شناسه تسک	عنوان فعالیت فنی	مسئول انجام	تخمین زمان (ساعت)	وضعیت نهایی
ST-01	راه‌اندازی مخزن Git، ساخت محیط مجازی (`.venv`) و پیکربندی ابزارها	امیرحسین میرزایی و رضا صفری	۲	تکمیل شد

شناسه تسک	عنوان فعالیت فنی	مسئول انجام	تخمین زمان (ساعت)	وضعیت نهایی
ST-02	طراحی معماری شی‌گرا، لایه دامنه (`TaskEntity`) و اینترفیس‌های انتزاعی طبق اصول SOLID	امیرحسین میرزایی	۴	تکمیل شد
ST-03	طراحی مدل داده ORM در SQLAlchemy و راه‌اندازی ارتباط پایگاه‌داده SQLite	امیرحسین میرزایی	۳	تکمیل شد
ST-04	پیاده‌سازی کلاس مخزن داده (`SQLiteTaskRepository`) و رعایت اصل LSP	امیرحسین میرزایی	۴	تکمیل شد
ST-05	پیاده‌سازی لایه سرویس بیزینس (`TaskService`) و تزریق وابستگی (DIP)	رضا صفری و امیرحسین میرزایی	۳	تکمیل شد
ST-06	تعریف اسکیمای Pydantic و اعتبارسنجی دقیق ورودی (جلوگیری از عنوان خالی و فاصله‌ها)	رضا صفری	۳	تکمیل شد
ST-07	پیاده‌سازی کنترلرها و اندپوینت‌های RESTful در FastAPI (اندپوینت‌های `POST /api/tasks` و `GET /api/tasks`)	رضا صفری	۳	تکمیل شد

شناسه تسک	عنوان فعالیت فنی	مسئول انجام	تخمین زمان (ساعت)	وضعیت نهایی
ST-08	پیاده‌سازی رابط کاربری فارسی با پالت سرمه‌ای/فیروزه‌ای/کهربایی و لبه‌های سه‌بعدی Specular	رضا صفری	۶	تکمیل شد
ST-09	اعتبارسنجی تعاملی کلاینت با `:user-invalid`، دسترس‌پذیری ARIA و اسکریپت ارتباط با API	رضا صفری	۵	تکمیل شد
ST-10	نگارش تست‌های خودکار واحد و یکپارچگی (Pytest) شامل ۸ تست پاس‌شده	رضا صفری و امیرحسین میرزایی	۴	تکمیل شد
ST-11	مستندسازی کدها با استاندارد Color Block و ساخت راه‌اندازهای خودکار (`start.bat` و `run.py`)	رضا صفری	۳	تکمیل شد
ST-12	برگزاری ۴ جلسه هماهنگی تیم، تدوین مستندات اسکرام (Review و Retrospective) و README.md	رضا صفری و امیرحسین میرزایی	۴	تکمیل شد

برنامه اجرا و هماهنگی میان اعضا

- **هفته اول:** تمرکز امیرحسین میرزایی بر طراحی مدل‌های دیتابیس در SQLAlchemy، پیاده‌سازی کلاس `SQLiteTaskRepository` و زیرساخت معماری SOLID؛ هم‌زمان تمرکز رضا صفری بر اعتبارسنجی دقیق داده‌های ورودی با Pydantic، سرویس بیزینس و اندپوینت‌های RESTful در FastAPI.
- **هفته دوم:** تمرکز رضا صفری بر توسعه رابط کاربری فرانت‌اند با پالت رنگی اختصاصی و سایه‌های سه‌بعدی Specular، اعمال استانداردهای Modern Web Guidance، کامنت‌گذاری Color Block و ساخت

اسکریپت‌های اجرای یک‌کلیکی (`start.bat` و `run.py`)؛ و همکاری مشترک رضا صفری و امیرحسین میرزایی در اجرای تست‌های خودکار با Pytest، تأیید پایداری دیتابیس و نهایی‌سازی مستندات.

معیارهای تکمیل (Definition of Done - DoD)

یک ویژگی زمانی «انجام‌شده» (Done) تلقی می‌شود که تمامی شروط زیر را پاس کرده باشد:

1. کد با معماری شی‌گرا و اصول پنج‌گانه SOLID تطابق داشته و دارای اینترفیس‌های انتزاعی باشد.
2. اعتبارسنجی ورودی مانع ورود عنوان خالی یا فواصل خالی (Whitespace) چه در کلاینت و چه در سرور شود.
3. داده‌ها در فایل دیتابیس محلی SQLite ذخیره شده و پس از ریستارت سرور سالم و بازیابی‌پذیر بمانند.
4. ۱۰۰٪ تست‌های خودکار واحد و یکپارچگی با Pytest (هر ۸ تست) بدون خطا پاس شوند.
5. تمامی فایل‌های کد پروژه بر اساس استاندارد Color Block Commenting کامنت‌گذاری و بازبینی (Code Review) شده باشند.
6. پروژه دارای اسکریپت راه‌انداز خودکار (`start.bat` و `run.py`) بوده و تغییرات با پیام‌های کامیت استاندارد در گیت ذخیره شده باشند.

۳. گزارش جلسات هماهنگی تیم (Sync Meetings)

در طول دوره دوهفته‌ای اسپرینت اول، ۴ جلسه هماهنگی کوتاه (۱۵ الی ۲۰ دقیقه) میان رضا صفری و امیرحسین میرزایی برگزار گردید:

جلسه ۱ (هفته اول – روز ۲): تصمیم‌گیری معماری و ساختار پروژه

- **پیشرفت:** مخزن Git راه‌اندازی شد، استک فنی FastAPI + SQLite تصویب گردید و ساختار دامنه (`TaskEntity`) و اینترفیس‌های انتزاعی (`ITaskRepository` و `ITaskService`) پیاده‌سازی شدند.
- **موانع:** چالش جلوگیری از وابستگی مستقیم لایه بیزینس به ORM مطرح بود.
- **تصمیمات:** به‌کارگیری الگوی مخزن (Repository Pattern) جهت جداسازی کامل دیتابیس از منطق بیزینس (تضمین اصل DIP و LSP). امیرحسین میرزایی مسئول مدل‌های داده و ریپازیتوری گردید؛ رضا صفری طراحی اسکیمای ورودی و ولیدیشن‌ها را عهده‌دار شد.

جلسه ۲ (هفته اول – روز ۵): لایه داده، اعتبارسنجی و اندپوینت‌های API

- **پیشرفت:** مدل دیتابیس و مخزن `SQLiteTaskRepository` توسط امیرحسین میرزایی پیاده‌سازی شد؛ اسکیمای Pydantic، قوانین اعتبارسنجی و اندپوینت‌های RESTful (اندپوینت‌های `POST /api/tasks` و `GET /api/tasks`) توسط رضا صفری راه‌اندازی شدند.
- **موانع:** عبور رشته‌های شامل صرفاً فاصله خالی (Whitespace) از اعتبارسنجی پیش‌فرض طول رشته.
- **تصمیمات:** افزودن اعتبارسنج سفارشی `@field_validator` در Pydantic توسط رضا صفری جهت حذف فضاهای خالی اطراف متن با متد `strip` پیش از ارزیابی طول.

جلسه ۳ (هفته دوم – روز ۲): رابط کاربری فارسی، پالت رنگی مدرن و دسترس‌پذیری وب

- **پیشرفت:** فرانت‌اند با استایل راست‌چین (RTL)، فونت Vazirmatn، پالت رنگی ۶گانه (سرمه‌ای کهکشانی `#0B132B` و `#1C2541`، آبی فولادی `#3A506B`، فیروزه‌ای `#5BC0DE`، کرم `#F4D6B0` و کهربایی `#D97706`) و لبه‌های سه‌بعدی Specular توسط رضا صفری پیاده‌سازی شد.
- **موانع:** ناخوانا بودن تاریخ میلادی برای کاربران فارسی‌زبان و لزوم نمایش وضعیت خالی بدون وظیفه (Empty State).
- **تصمیمات:** اعمال استانداردهای Modern Web Guidance (شبه‌کلاس `:user-invalid`، هماهنگی ویژگی‌های ARIA، نمایش پیام وضعیت خالی و تبدیل تاریخ به خورشیدی با `Intl.DateTimeFormat`) توسط رضا صفری.

جلسه ۴ (هفته دوم – روز ۵): تست‌های خودکار، کامنت‌گذاری Color Block و اجرای یک‌کلیکی

- **پیشرفت:** ۸ تست خودکار در Pytest نوشته شد و با موفقیت ۱۰۰٪ پاس گردید؛ پایداری داده‌ها در SQLite توسط امیرحسین میرزایی تأیید شد و تمام ۱۵ فایل پروژه بر اساس استاندارد Color Block توسط رضا صفری کامنت‌گذاری شدند.
- **موانع:** در برخی سیستم‌های ویندوزی، پورت پیش‌فرض 8000 در بازه پورت‌های رزروشده سیستم‌عامل (WinNAT/Hyper-V) قرار داشت و اجرای دستی دستور `uvicorn` را با چالش مواجه می‌کرد.

- **تصمیمات:** طراحی اسکریپت هوشمند `run.py` (برای تشخیص خودکار اولین پورت آزاد سیستم و باز کردن مرورگر) و فایل `start.bat` (برای نصب خودکار محیط مجازی و اجرای یک‌کلیکی پروژه در ویندوز) توسط رضا صفری و نهایی‌سازی مستندات اسکرام.

۴. گزارش جلسه بازبینی اسپرینت (Sprint Review)

- **تاریخ برگزاری:** پایان هفته دوم اسپرینت اول
- **اعضای حاضر:** رضا صفری و امیرحسین میرزایی
- **قابلیت‌های تکمیل‌شده در اسپرینت اول:**
  a. **ثبت وظیفه جدید (PB-01 — مسئول: رضا صفری):** فرم ثبت وظیفه واکنش‌گرا و راست‌چین با فیلدهای عنوان و توضیحات اختیاری، تخصیص وضعیت اولیه «انجام‌نشده» و زمان ثبت، و پاک‌سازی خودکار فرم پس از ارسال.
  b. **نمایش فهرست وظایف (PB-03 — مسئول: رضا صفری):** نمایش کارت‌های وظایف با شناسه یکتا (#ID)، عنوان، توضیحات، وضعیت و تاریخ خورشیدی، مرتب‌سازی نزولی، طراحی بصری با پالت سرمه‌ای/فیروزه‌ای/کهربایی و لبه‌های سه‌بعدی Specular، و کامپوننت Empty State.
  c. **اعتبارسنجی دوطرفه ورودی‌ها (PB-02 — مسئول: رضا صفری):** جلوگیری از ثبت عنوان خالی یا فاصله در کلاینت با `:user-invalid` و ARIA و در سرور با Pydantic و کد خطای 422.
  d. **ماندگاری داده‌ها در پایگاه داده (PB-04 — مسئول: امیرحسین میرزایی):** ذخیره‌سازی پایدار در فایل `todo.db` با SQLite و SQLAlchemy و بازیابی کامل اطلاعات پس از ریستارت سرور.
  e. **کیفیت مهندسی، تست و راه‌اندازی خودکار (همکاری مشترک):** پوشش ۸ تست خودکار در Pytest با موفقیت ۱۰۰٪، کامنت‌گذاری کامل Color Block و اجرای یک‌کلیکی با `start.bat` و `run.py`.
- **میزان تحقق هدف اسپرینت: ۱۰۰٪** اهداف اسپرینت اول بر اساس معیارهای DoD محقق شد.
- **بازخوردهای دمو و پیشنهادات اسپرینت دوم:**
  - پیاده‌سازی **PB-05** (تغییر وضعیت انجام با متد PATCH و خط‌خوردگی متن) — مسئول: رضا صفری.
  - پیاده‌سازی **PB-06** (حذف و ویرایش وظیفه در سمت سرور و دیتابیس) — مسئول: امیرحسین میرزایی.
  - پیاده‌سازی **PB-07** (فیلتر و جستجوی زنده وظایف) — مسئول: رضا صفری.

۵. گزارش جلسه بازنگری عملکرد (Sprint Retrospective)

- **۱) چه کارهایی به‌خوبی انجام شدند؟**
  - تفکیک شفاف لایه‌ها با اصول پنج‌گانه SOLID توسط امیرحسین میرزایی.
  - طراحی رابط کاربری چشم‌نواز و دسترس‌پذیر با پالت سرمه‌ای کهکشانی، فیروزه‌ای و کهربایی، لبه‌های سه‌بعدی Specular و استانداردهای Modern Web Guidance توسط رضا صفری.
  - کامنت‌گذاری مهندسی Color Block، پوشش ۸ تست خودکار در Pytest و طراحی راه‌انداز هوشمند `start.bat` و `run.py` برای اجرای بدون دردسر روی تمام سیستم‌ها.
- **۲) چه مشکلاتی در طول اسپرینت وجود داشت؟**
  - چالش اولیه در اعتبارسنجی کاراکترهای فاصله خالی (Whitespace) در ولیدیتور پیش‌فرض طول رشته.
  - رزرو بودن پورت 8000 توسط سرویس‌های شبکه ویندوز در برخی سیستم‌ها و اجرای دستی تست‌ها در اوایل اسپرینت.
- **۳) ریشه اصلی مشکلات چه بود؟**
  - رفتار پیش‌فرض توابع اعتبارسنجی طول رشته بدون فراخوانی متد `strip`، محدودیت بازه پورت‌های رزرو TCP در ویندوز، و عدم پیکربندی اولیه هوک‌های گیت در روز اول.
- **۴) اقدام اصلاحی قطعی برای اسپرینت دوم (Action Item):**

مؤلفه	شرح مصوب
اقدام اصلاحی قطعی (Action Item)	تعریف و ثبت یکپارچه قراردادهای رابط کاربری و API (API Contract & Mocking) در ابتدای جلسات برنامه‌ریزی اسپرینت دوم، به همراه طراحی اسکریپت اجرای خودکار آزمون‌های رگرسیون (`run_tests.bat`) قبل از ادغام هر شاخه در گیت.
مسئولین پیگیری و اجرا	رضا صفری و امیرحسین میرزایی

مؤلفه	شرح مصوب
معیار پذیرش و موفقیت	عدم بروز هرگونه ناهماهنگی در ساختار داده‌های مبادله‌شده میان فرانت‌اند و بک‌اند، و اجرای موفق ۱۰۰٪ تست‌های واحد و یکپارچگی پیش از ثبت هر کامیت در مخزن پروژه.

۶. ساختار معماری و رعایت اصول پنج‌گانه SOLID

کد نرم‌افزار بر اساس معماری لایه‌ای تمیز (Layered Clean Architecture) در پوشه `app` ساماندهی شده و تمامی اصول پنج‌گانه SOLID در آن رعایت گردیده است:

1. **Single Responsibility Principle (SRP):** تفکیک کامل مسئولیت‌ها میان ماژول‌ها؛ صرفاً برای `schemas`، صرفاً برای ساختار جدول دیتابیس `models`، صرفاً `sqlite_repository` برای اعتبارسنجی ورودی/خروجی، صرفاً `task_router` برای مدیریت درخواست‌های HTTP.
2. **Open/Closed Principle (OCP):** تعریف اینترفیس‌های انتزاعی (`ITaskRepository` و `ITaskService`) به‌گونه‌ای که سیستم برای افزودن قابلیت‌ها یا تغییر دیتابیس در اسپرینت‌های بعدی باز و در برابر دست‌کاری کدهای قبلی بسته باشد.
3. **Liskov Substitution Principle (LSP):** قابلیت جایگزینی کامل `InMemoryTaskRepository` با `SQLiteTaskRepository` بدون کوچک‌ترین تغییر در لایه سرویس (که به صورت عملی در تست‌های خودکار `tests/test_tasks.py` آزموده شده است).
4. **Interface Segregation Principle (ISP):** طراحی اینترفیس‌های شفاف، کوچک و متمرکز در `app/domain/interfaces.py` بدون تحمیل متدهای اضافی به کلاس‌های پیاده‌ساز.
5. **Dependency Inversion Principle (DIP):** تزریق وابستگی اینترفیس‌ها در `app/api/dependencies.py` با مکانیزم FastAPI `Depends` در فایل به‌طوری که لایه‌های سطح بالا به پیاده‌سازی مستقیم دیتابیس وابسته نیستند.

۷. راهنمای اجرا و راه‌اندازی سریع پروژه

برای سهولت اجرا بر روی تمامی سیستم‌عامل‌ها و جلوگیری از تداخل پورت‌ها، دو روش راه‌اندازی در پروژه تعبیه شده است:

روش اول: اجرای یک‌کلیکی در ویندوز (پیشنهادی)

کافی است روی فایل start.bat در پوشه اصلی پروژه دبل‌کلیک کنید (یا در ترمینال دستور start.bat را وارد نمایید). این فایل به صورت کاملاً خودکار:

1. محیط مجازی پایتون (`venv.`) را در صورت وجود نداشتن می‌سازد و کتابخانه‌های `requirements.txt` را نصب می‌کند.
2. اسکریپت هوشمند `run.py` را فراخوانی می‌کند تا اولین پورت آزاد سیستم (مانند 8000 یا 8100) را پیدا کند.
3. سرور را راه‌اندازی کرده و صفحه وب برنامه را به صورت خودکار در مرورگر باز می‌کند.

روش دوم: اجرای چندسکویی از طریق ترمینال (ویندوز، لینوکس و مک)

- **گام اول — ساخت محیط مجازی و نصب وابستگی‌ها:**
  - دستور ساخت محیط مجازی: `python -m venv .venv`
  - دستور نصب پکیج‌ها: `pip install -r requirements.txt`
- **گام دوم — اجرای هوشمند سرور و باز شدن خودکار مرورگر:**
  - دستور اجرا: `python run.py`
- **آدرس‌های دسترسی پس از اجرا:**
  - **رابط کاربری وب:** آدرس دقیق پورت انتخاب‌شده در ترمینال نمایش داده می‌شود و خودکار در مرورگر باز می‌گردد (پیش‌فرض: `http://127.0.0.1:8000` یا `http://127.0.0.1:8100`).
  - **مستندات تعاملی API (Swagger UI):** در مسیر `/docs` روی همان پورت (برای مثال `http://127.0.0.1:8100/docs`) در دسترس است.

نحوه اجرای تست‌های خودکار (Automated Tests)

برای اجرای مجموعه ۸ تست خودکار واحد و یکپارچگی با فریم‌ورک Pytest، دستور زیر را در ترمینال وارد نمایید:

- در محیط مجازی فعال: `pytest -v`
- در ویندوز بدون نیاز به فعال‌سازی محیط مجازی: `venv\Scripts\python -m pytest -v.`

(نتیجه اجرا: هر ۸ تست خودکار شامل تست‌های اعتبارسنجی، سرویس، اندپوینت‌ها و ماندگاری دیتابیس با موفقیت ۱۰۰٪ پاس می‌شوند).